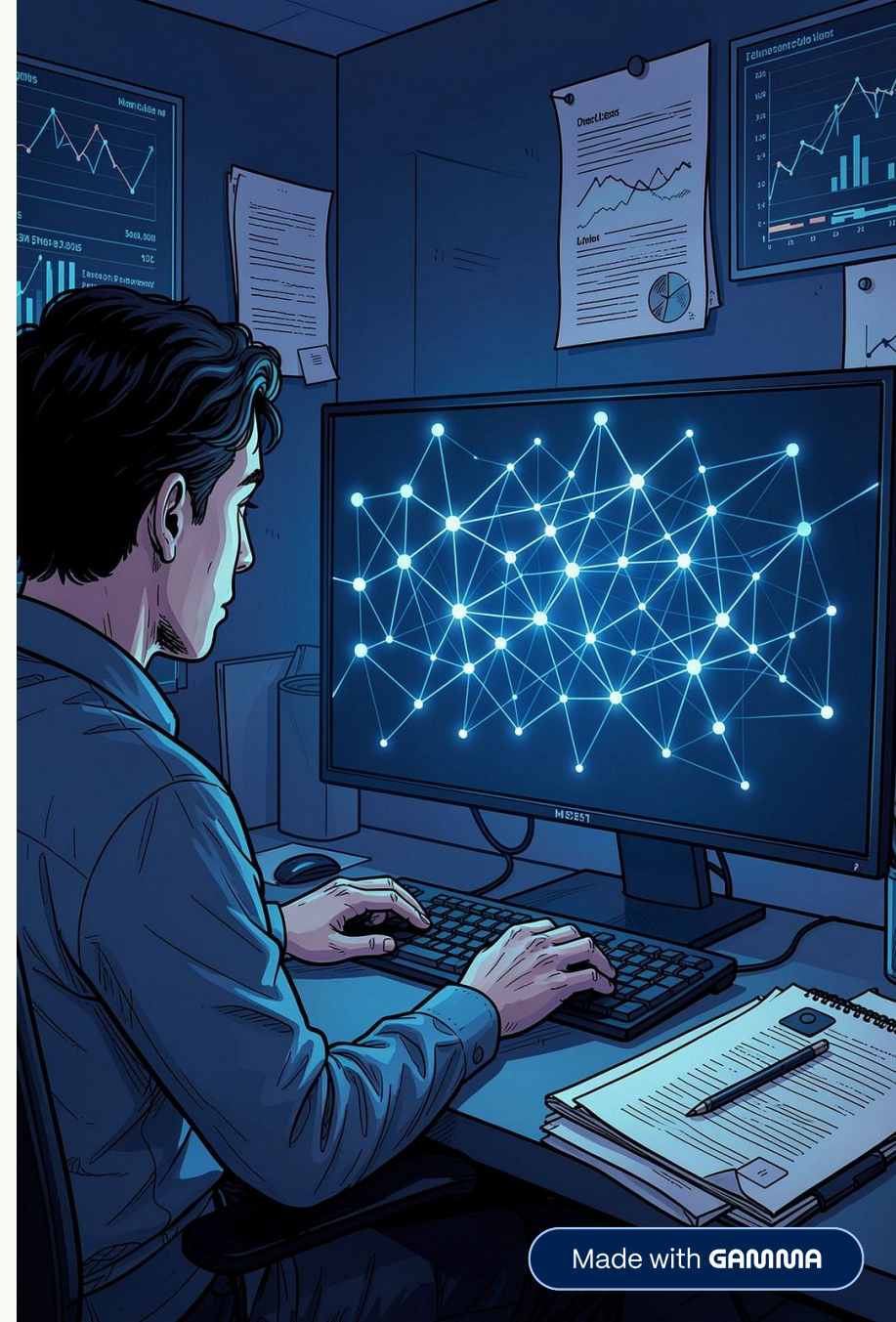


Smart Research Paper Recommendation System

Final Implementation, Testing & Results — Advanced Data Structures & Algorithms · Mini Project

Akula Yeshwanth (2510030357) · SRK Akshith (2510030365) · Parimal (2510030368) · Hemanth (2510030326)



What We Delivered

System Integration

Full end-to-end pipeline connecting data ingestion, DSA engine, and JavaFX GUI

GUI & Visualization

Interactive JavaFX interface with graph-based recommendation visualization

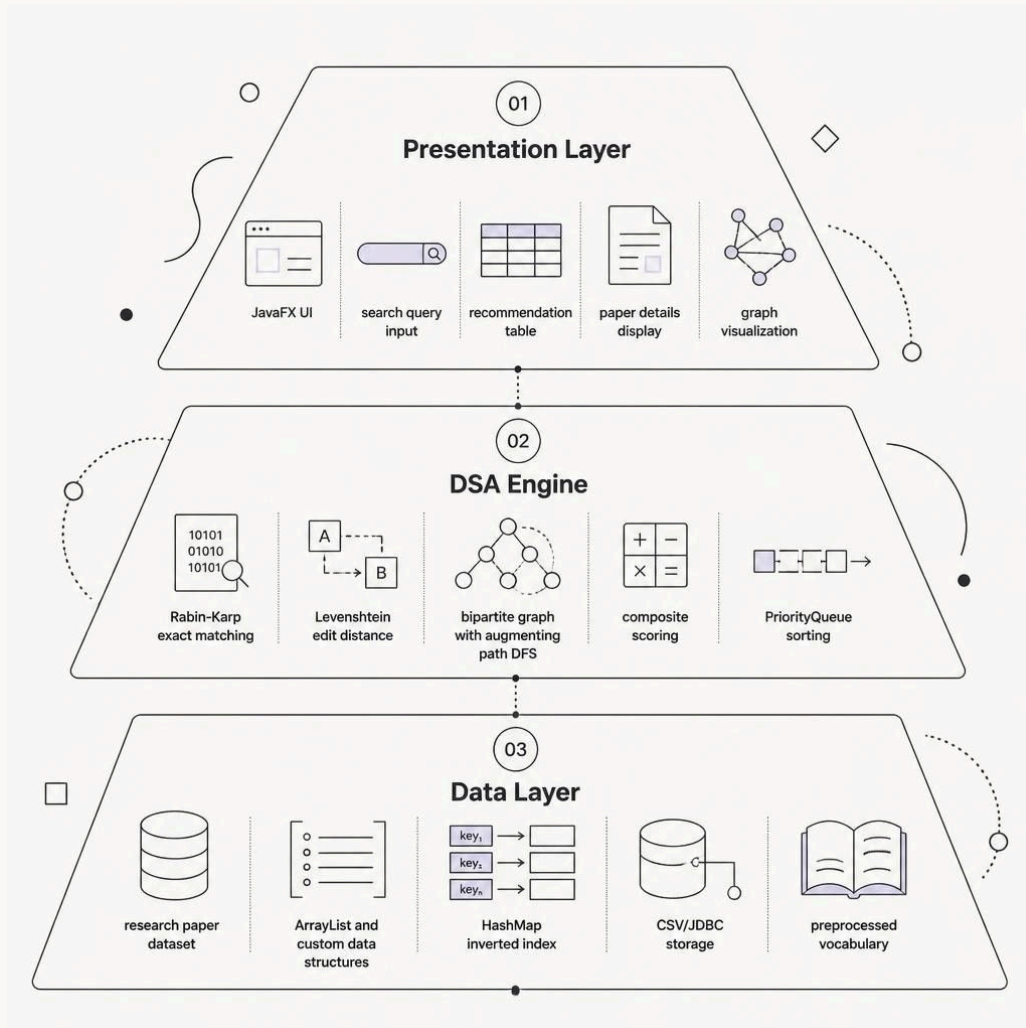
Testing & Evaluation

Functional test cases, performance benchmarks, and recommendation result analysis

Documentation

Limitations, future enhancements, and final project report

Three-Layer System Architecture



Presentation Layer

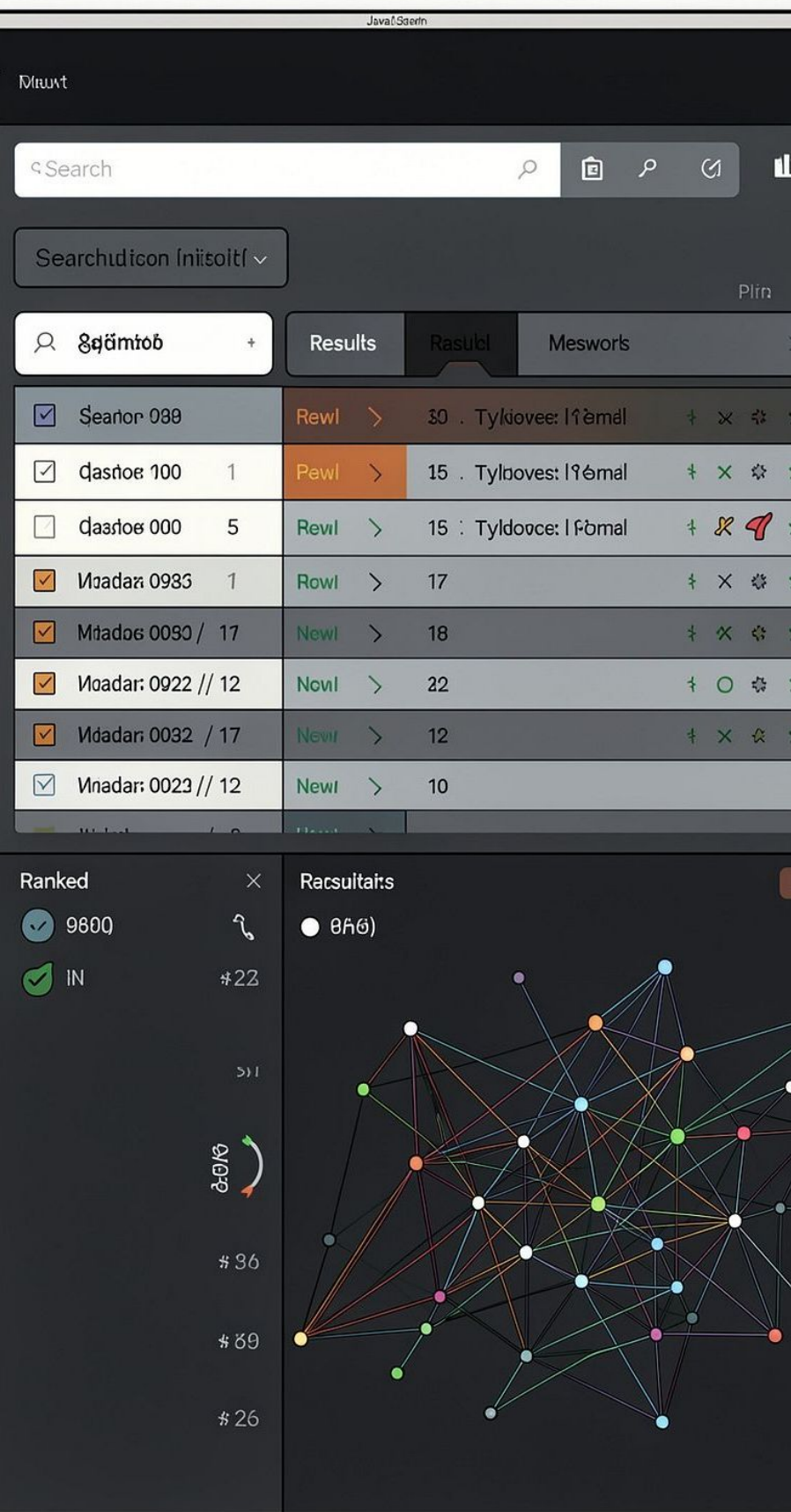
JavaFX interface for query input, ranked TableView output, and interactive graph visualization of topic-paper relationships.

DSA Engine

Core algorithmic layer — Rabin-Karp, Levenshtein DP, bipartite matching, and composite scoring run here.

Data Layer

Paper dataset stored via HashMap inverted index, ArrayList structures, and CSV/JDBC persistence.



USER INTERFACE

JavaFX GUI Implementation



Search & Query Input

Search bar supports multi-topic queries with a configurable fuzzy matching threshold



Ranked TableView

Displays paper title, author, matched terms, and composite relevance score per result



Graph Visualization

Visualizes topic-to-paper relationships, making bipartite matching output interpretable



Relevance Breakdown

Detailed per-field score display (title, keywords, abstract) for explainable ranking

End-to-End System Workflow

01

Query Preprocessing

Tokenization, normalization, and cleaning of user input

02

Rabin-Karp Search

Locate exact keyword occurrences across the paper dataset

03

Levenshtein DP

Detect approximate and fuzzy matches for term variations

04

Relevance Scoring

Combine exact and fuzzy scores into a composite relevance value

05

Bipartite Graph Construction

Build Topic ↔ Candidate Paper edges for multi-topic coverage

06

Maximum Bipartite Matching

Apply augmenting path DFS to maximize topic-paper assignments

07

Ranking & Output

PriorityQueue for Top-K selection; display ranked papers with relevance details

Functional Testing Strategy

Test Case	Expected Outcome
Exact keyword query	Relevant papers ranked highly
Misspelled keyword	Similar terms detected via Levenshtein
Multi-topic query	Diverse topic areas represented
Irrelevant keyword	Low composite relevance score
Varied fuzzy threshold	Recommendation set adjusts accordingly
Multiple candidate papers	Highest-scoring papers prioritized

Validation Checklist

- Rabin-Karp match correctness
- Levenshtein distance accuracy
- Graph edge assignments
- Composite score calculation
- Ranking order integrity
- GUI output consistency

Multi-Topic Query in Action

1

Three topics identified and preprocessed independently

2

Rabin-Karp finds exact matches; Levenshtein catches variations

3

Topic Paper edges built; augmenting path DFS applied

4

Composite scores computed; ranked list returned to user



Composite Scoring & Computational Complexity

Composite Relevance Formula

Final Score = 0.70 × Exact Match + 0.30 × Fuzzy Match

- Title weight: **50%**
- Keywords weight: **30%**
- Abstract weight: **20%**

PriorityQueue delivers Top-K in **$O(N \log K)$** ; full sort via QuickSort/MergeSort in **$O(N \log N)$** .

Module	Complexity
Dataset Ingestion	$O(N \times L)$
Rabin-Karp (avg)	$O(N + M)$
Rabin-Karp (worst)	$O(N \times M)$
Levenshtein DP	$O(M \times N)$
Bipartite Matching	$O(V \times E)$
Top-K PriorityQueue	$O(N \log K)$

❏ Observed multi-topic execution latency: **< 50 ms** — pure Java, no AI library dependency.

Results, Advantages & Limitations

Key Advantages

- **Pure Java** — no heavy neural model dependencies
- **Fast & typo-tolerant** — Rabin-Karp + Levenshtein DP
- **Multi-topic coverage** — bipartite graph matching
- **Explainable ranking** — deterministic, traceable scores
- **Interactive UI** — JavaFX with live graph visualization

Current Limitations

- Levenshtein DP scales poorly on long strings
- String similarity lacks deep semantic understanding
- Recommendation quality tied to dataset quality
- No personalization from individual user behavior
- Scoring weights require manual configuration

Future Scope & Conclusion

Scale Up

Support larger datasets with optimized indexing and reduced Levenshtein memory footprint

Semantic Depth

Integrate semantic similarity, citation-network analysis, and publication metadata

Personalization

Introduce user behavior learning and personalized recommendation profiles

Web Deployment

Migrate to a web-based interface with persistent cloud and database storage

- ✅ **Final Outcome:** A pure Java, DSA-driven research paper recommendation system — fast, diverse, and fully explainable, built entirely on advanced algorithms without external AI dependencies.

